Stage 4. Rates

Holidays, seasons, time-of-use, schedules, rate schedules, meter assignments and rate tests

PREPARED BY Energy Hippo implementation team
STATUS Being reworked (November 9, 2026). Do not enter yet

Stage 4 builds Northlake's whole rate library by hand in the Rate Modeler and Schedule screens: the shared holiday, season and time-of-use catalog; the nine Schedule bundles that combine them; twenty-five rate schedules in total, sixteen covering every base provider tariff (Ridgeline Wind's three among them) and nine built from the dream-only entries in data/eem/northlake-rates-v1.yaml, that between them fire all nine kinds of billing determinant; the meter-to-rate assignments that put those rates to work; and the Test Rates runs that prove each one calculates a bill before a single real bill is ever generated. It depends on Stage 1's six utility providers and three companies, and on Stage 2's meters existing well enough to be picked by name in the Tenant Rebill tab (Stage 2 is being written in parallel; meters are referred to here by their scenario id and nothing in this stage waits for them to be typed in first).

Rework in progress: read this first

Stage 4 was reworked on November 9, 2026 and this chapter has not caught up. **Do not enter Stage 4 from the steps below.** The YAML files are ahead of the text:

- **Real tariffs, entered as they read.** 05-rates.yaml now holds the six providers' 16 tariffs the way their own tariff sheets and bills state them: Valley Electric's three time-of-use periods, the percentage lines its bills and Sierra Gas's carry, River City's meter and line sizes as rate options, per-meter prices for lighting fixtures, stormwater units and the plant's fixed share, the Helios escalator, and a 2026 price step.
- **Schedules.** Holidays run 2024 to 2027. There are eight schedules: all hours, Valley Electric's five periods and the two gas seasons (01, 03, 04).
- **Assignments.** Every real meter carries its provider's tariff: 56 meters, 74 rows (06-rate-assignments.yaml).
- **Rate tests.** 26 runs, each with its total and lines worked out beforehand (07-rate-tests.yaml).
- **Rate Lab.** The nine "NL-" feature rates moved out of the real providers into 08-rate-lab.yaml, the starting material for a seventh provider that will carry the full rate engine. It is not ready to enter.

Why, what is still open, and the scenarios planned for Stage 12 are in the coverage review. Steps 1 to 7 below describe the rates as they were before the rework and will be rewritten when the Rate Lab is done.

Before you start

- Stage 1 and Stage 2 checks are green: six providers, three companies, and the meters this stage assigns rates to.
- Toggle "Show all providers" the first time the Rate Modeler tree opens. It defaults to providers that already have a rate, and none of Northlake's six do yet.
- Two rows already exist on this installed database from an earlier setup step: UtilitySchedule 1 ("VED-TOU-GS", pointed at a placeholder season and time-of-use) and UtilityDeterminantScript 1 ("Round

Bill Item Down 2 Decimals"). This stage renames and repurposes the first and reuses the second as-is; neither needs to be deleted and recreated.

- Enter everything through the applications. If a screen cannot take a value this stage needs, record it under Findings at the end of this chapter. Do not fill the gap with SQL.

Step 1. Holidays

Where: Database Administrator, Schedule, Holidays tab.

Enter: 01-holidays.yaml, one holiday set.

Order inside the editor: Add the holiday set on the combo next to Selected Holiday, type its Holiday Set Name, save, then Add Holiday for each of its twelve date rows.

EEM creates for you: HolidayID 0, "No Holidays", already exists as the system default and needs no entry; it stays the choice for every Schedule that should not skip a day.

Record: the Holiday Set's generated id.

Check: `holidays_exist`.

Step 2. Seasons

Where: Database Administrator, Schedule, Seasons tab.

Enter: 02-seasons.yaml, four seasons (one of them a repurpose of an existing row).

Order inside the editor: SeasonID 1, "New Season," already exists with one Jan 1-Dec 31 date. Select it, rename it to VED Summer (Jun-Sep), delete its date, and add the real Jun 1-Sep 30 date in its place -- Season Type is locked once a season has dates, so its Month Day type carries over unchanged. Then Add Season three more times for VED Winter, SGU Winter and SGU Summer, each with its own one date row.

Record: each season's generated id (three new; SeasonID 1 keeps its id).

Check: `seasons_and_tou_exist`.

Step 3. Time of use

Where: Database Administrator, Schedule, Time of Use tab.

Enter: 03-time-of-use.yaml, four time-of-use definitions (one of them a repurpose of an existing row).

Order inside the editor: UtilityTOUID 1, "Peak Hours," already exists. Select it, rename it to Summer On-Peak (Wkdy 12-18), and repaint its grid: All Off, then Fill Monday-Friday for hours 12 through 17. Then Add three more: Summer Off-Peak (All On, then Clear the same block), Winter On-Peak (Wkdy 17-21), and Winter Off-Peak (All On, then Clear that block). UtilityTOUID 0, "All Hrs," is the system default and needs no entry.

Record: each definition's generated id (three new; UtilityTOUID 1 keeps its id).

Check: `seasons_and_tou_exist`.

Step 4. Schedules

Where: Database Administrator, Schedule, the left "Schedules" list.

Enter: 04-schedules.yaml, nine schedules (one a rename of the existing ScheduleID 1).

Order inside the editor: pick a schedule from the left list, then use each of the three tabs' own Selected Season / Selected Holiday / Selected Time of Use combo to assign that piece to the schedule currently selected -- the app's own hint says this exactly: choose a schedule first, then use each tab to pick what it uses. Rename ScheduleID 1 from "VED-TOU-GS" to "Summer On-Peak" (a Schedule name and a Rate name are different things, and leaving it named after a rate is confusing once it is reused across the whole library) and point its three tabs at VED Summer, Northlake Holidays and Summer On-Peak. Add the other seven: Summer Off-Peak, Winter On-Peak, Winter Off-Peak (all three with Northlake Holidays), Summer (Demand), Winter (Demand), Gas Winter and Gas Summer (all four with No Holidays). ScheduleID 0, "Annual All Hrs," is the system default.

Record: each new schedule's generated id.

Check: `schedules_exist`.

Step 5. Rates

Where: Database Administrator, RateModeler, Rate Edit.

Enter: 05-rates.yaml, four scripts (three new, one reused) and twenty-five rate schedules.

Order inside the editor: on RateModeler's Scripts screen, Add Script for the three new ones (Description, Type Pre/Post, SQL in the syntax editor); the fourth, rounding, already exists and needs no entry. Then, per provider: Add Rate (name only at first save), fill Currency and Measure Type before either locks behind a Determinant, and use the class All that Add Rate creates (the always-applied class; Add Rate Class only for NL-OPTIONS-Phase's option classes, which are Unique -- see Findings), Add Determinant under the class (Charge Name, Charge Type, Units, Schedule, Significance, scripts, toggles), and Add Effective Date under the determinant for each value the tariff carries. A Charge Type 200 or 210 determinant's tiers or blocks are entered on that Effective Date, not as separate determinants. A demand determinant's Ratchet grid (its own Add Ratchet button) only applies on NL-LG-Ratchet.

Sixteen base provider rates come straight from `data/providers/*.yaml`; the 2025 rate step Valley Electric's tariff carries is a second Effective Date on each affected determinant, not a second rate. **Ten dream-only schedules** come from `data/eem/northlake-rates-v1.yaml`; nine of them are new rates and the tenth, NL-MULTI-Commodity, is folded into the base RCU and NTP multi-commodity rates it describes rather than becoming an eleventh rate.

Record: each script's and rate's generated id.

Check: `rates_by_provider`, `determinant_kinds_covered`, `rate_classes`, `effective_dates_cover_2024_2025`, `determinant_blocks`, `scripts_exist`, `ridgeline_rates`.

Step 6. Rate assignments

Where: Database Administrator, Nodes. Select a meter node, open its Definitions dock, and use the Tenant Rebill tab.

Enter: 06-rate-assignments.yaml, one record per provider-billed meter across the three companies (fifty-six records, two of them Ridgeline Wind's).

Order inside the editor: pick the Bill Scenario at the top of the tab (every Northlake company has only "Actual" today), then use the Rate Config grid's + Rate Assignment button for each rate the meter carries, picking the rate, a Start Date, and leaving End Date open unless the meter's assignment genuinely ends. A rate only appears in the combo once it is on the meter's own utility provider. Student Center's shadow RTP row needs a second Bill Scenario, "Scenario (what-if)," created first on its company's Bill Scenarios page (Stage 1 scope, not repeated here).

Why this is last before testing: every metered account needs its rate in place before `rp_RateComparison` has anything to shadow-bill.

Record: nothing new; the meter node itself is the key.

Check: `meter_assignments`, `ridgeline_assignments`.

Step 7. Rate tests

Where: Database Administrator, RateModeler, Test Rates.

Enter: 07-rate-tests.yaml, one test run per rate schedule that predates Ridgeline Wind (twenty-two runs) and seven runs that reproduce the Ridgeline-era itemized bills (twenty-nine in all).

Order inside the editor: Test Configuration (Rate to Test, Start/End Date, Energy/Demand Multiplier left at 1.0000, Rate Classes -- Select All except where a run calls out specific classes), then Intervals (every run here uses Generate Reads, since none of Stage 2's meters exist yet to Load from Point, except the last seven, which paste a bill's exact quantities), then Results, where the Bill Total and its charge lines appear.

Where the numbers come from: a handful of rates are pure fixed charges with no usage-based determinant (VED-AL-1, RCU-SW-ERU, RCU-FIRE), so their Bill Total is exact regardless of the generated reads and is given in the file. Everywhere else there is no independent number to check against without real interval data, so the file says to record the run's own total and verify the specific mechanic the run exists to prove (a ratchet floor, a tiered minimum bill, a stepped-demand cap, and so on). The seven Ridgeline-era runs are the exception: the itemized bills give every quantity and amount, so each of those runs states its lines to the cent.

A run's storage is not a database row. Test Rates scenarios save as JSON files under `%AppData%\EEMSuite\RateEngineTestingScenarios\`, one per run (verified in `RateTestScenarioFileStore`); `checks.sql`'s `rate_test_runs` line says so instead of selecting from a table that does not exist.

Record: nothing; note in the Test Rates dashboard which runs are saved.

Check: `rate_test_runs` (manual, see above).

Ridgeline Wind

When the Northlake Thermal Plant bought Ridgeline Wind on 2024-07-01, Stage 4 gained three rate schedules, four rate assignments on three meters and seven Test Rates runs (added 2026-10-01). No holiday, season, time-of-use or schedule bundle is new: every Ridgeline determinant uses Annual All Hrs.

Enter:

- The last three records of 05-rates.yaml. VED-GEN-IC (the customer charge, station service energy, the generation credit and the utility users tax on the first two) and VED-BCT (the $12.50 administrative fee) go under Valley Electric; RWL-XFER-2024 (transfer energy and the research credit) goes under the Ridgeline Wind provider that Stage 1 creates. VED-TOU-PRI also gains the public purpose program and utility users tax lines that the Central Plant's itemized bill carries.
- RWL-XFER-2024's two determinants go under the class All that Add Rate creates, which the engine applies to every meter, and each gets a Commodity Mapping (Transfer Energy to Non-TOU kWh for its quantity and its charge, Research Credit to Elec Misc Chgs/Credits). The engine turns an amount into a bill line only through the mapping; without it the generated statement would have no lines.
- In 06-rate-assignments.yaml, the Ridgeline interconnection meter's two rows (VED-GEN-IC and VED-BCT), a VED-BCT row on the Central Plant's meter and one RWL-XFER-2024 row on the Ridgeline Farm Transfer Meter, the logical billing meter that Stage 2 adds, all from 2024-07-01. The ten turbine pad meters get no row.
- The last seven runs in 07-rate-tests.yaml.

Where the numbers come from: the provider files give the rates and the four itemized bills in `data/bills/northlake-itemized-bills-v1.yaml` give the quantities. Each run pastes a bill's own quantities and sorts the bill's lines into the ones the engine computes and the ones that come from outside it, so adding the two back up reproduces the bill: $1,077.45 and $6,753.10 for the generator account's two bills, and $129,330.27 for Ridgeline's hand-made July 2024 statement, which the rate matches to the cent. The Central Plant's $151,171.47 bill does not come out of the engine, and its run says by how much and why.

What the engine cannot do with these rates is the Findings below: it cannot price one channel of the interconnection meter, bill reactive energy, move the generator's credit to the Central Plant, run VED-BCT beside a base rate, or generate one statement from ten meters (the billing meter is the way round for the last).

Check: `ridgeline_rates`, `ridgeline_assignments`.

Stage 4 is done when

- All nine `UtilityDeterminantType` kinds (100 RTP, 200 Charge Period, 210 Charge Period Fixed Value, 300 Fixed By Day, 400 Fixed By Period, 500 Min Value, 600 Max Value, 700 Bill Adj, 800 Taxes) have fired in at least one Test Rates run.
- Every one of the sixteen base provider schedules (Ridgeline Wind's three among them) and ten dream schedules exists in Rate Modeler.
- Every metered account across the three installed companies (Valley Electric, Sierra Gas, River City, Northlake Thermal Plant, Helios Onsite Solar and Ridgeline Wind) has a Tenant Rebill rate assignment, except NL-STEP-Demand's Town Center meter and NL-MULTI-Currency's cross-border meter, neither of which exists yet, and Ridgeline's ten turbine pad meters, which the Ridgeline Farm Transfer Meter bills for them.
- RWL-XFER-2024's determinants are under its All class and both have a Commodity Mapping, so the Ridgeline Farm Transfer Meter can be priced and its statement written (`ridgeline_rates`).

- Each of the seven Ridgeline-era runs shows the engine lines its record gives, to the cent: the generator, VED-BCT and Ridgeline runs match the bill lines they name, and the Central Plant run shows the overstatement its record explains.
- `rp_RateComparison` can shadow-bill Student Center: an Actual VED-TOU-GS assignment and a Scenario (what-if) NL-RTP-Index assignment both exist on the same meter over the same period.

Run `checks.sql` beside this chapter against the instance database to see each of these as a result set (all but `rate_test_runs`, which is verified by hand as the check itself explains).

Findings

Two things carried forward from the dream-customer entry-path research, both confirmed against the current code while entering this stage:

- **Rate Modeler never shows a class's type (finding 8).** Add Rate creates the class All with the always-applied type (`UtilityRate_Save` inserts it as ClassType 0, and the desktop saves with FromImport 0), and every class added after it is Unique (`UtilityRateClass_Save` inserts ClassType 1), an option class. The engine applies All to every meter and an added class only as its Rate Option's default or a meter binding (`UtilityRate_Exec`), and the option editor never offers All (`RateViewModel`). Nothing on screen tells the two apart, so this packet's source took All for Unique and asked for an always-applied class for NL-OPTIONS-Phase's and NL-RES-Tiered's tax and minimum-bill scoping; All already is that class, and every rate here puts its determinants under it. A determinant put under an added class that is no option's default prices nothing on a generated bill and Test Rates does not show it (finding 69).
- **Nothing binds one meter to one rate-class option.** NL-OPTIONS-Phase can define Rate Options (Min/Max classes, a default) on the Rate screen, but the meter-to-rate assignment (`TenantRateAssignmentModel`, the Tenant Rebill grid) carries only a Rate, Start Date, End Date and a Bill checkbox -- no `RateClassId` or `RateClassOptionId`. The domain and Rate Import/Export JSON path both support per-meter rate-class options (`MeterRateClassOption`, `UtilityRateClassMeter`), so the data model is ready; only an interactive screen to set it is missing. NL-OPTIONS-Phase's Test Rates run in 07-rate-tests.yaml is where this is exercised: every class can be tested at once, but no meter can be told to bill under just one.

Two unsupported billing constructs are modeled as synthetic stand-ins with scripts, both documented on their own rate in 05-rates.yaml:

- **Power factor** has no native math in the rate engine. NL-LG-Ratchet's demand determinant carries a Post Script that adds a flat reactive-power adjustment instead of computing one from a real power-factor reading (Northlake has no PF point).
- **Base/excess contract demand splitting** is not a built-in determinant behavior. NL-CONTRACT-2Part's Base Contract Demand and Excess Demand determinants use a matched Pre/Post script pair to cap one and bill the remainder on the other.

Coincident peak, critical-peak pricing and net-metering banking are not separately modeled: Helios's Net Metering Credit determinant is an ordinary negative-value usage charge, not a real banked-balance construct, and no CPP or coincident-peak rate exists in this library at all (nothing in the source documents called for one).

One structural gap found while building the flagship rate: the source entry plan for NL-GS-TOU-Seasonal never assigns a determinant to the winter on-peak schedule it otherwise creates, leaving the tariff's winter weekday 5-9 PM window unpriced. This stage adds an Energy On-Peak (Winter) determinant to close it, with an assumed value noted on that determinant.

Two smaller, screen-level findings from entering the base provider tariffs: neither Electricity nor Potable Water's unit list offers a per-fixture or per-ERU countable unit, so VED-AL-1's area lighting and RCU-SW-ERU's stormwater fee are both entered as a flat monthly amount (the published per-unit rate times an assumed fixture/ERU count) rather than a true per-unit charge. Both are noted on their own rate.

Ridgeline Wind's three rates met five more limits of the rate engine, each confirmed against the current code (the evidence is on the rate in 05-rates.yaml):

- **A determinant cannot read one channel of a bidirectional meter (finding 62).** A usage determinant takes every point under the meter whose unit class is the measure type's usage class and adds them (`UtilityRates_GetPoints`, `UtilityRates_LoadDataValues`), and the only per-point switch, Tenant Rebill's Point Assignment, drops a point from every rate in a bill scenario. The interconnection meter's Exported kWh and Station Service kWh are both kWh, so VED-GEN-IC's Station Service Energy and Generation Credit each price the sum of the two. Both stay as the tariff states them (the negative per-kWh value itself is fine), and a Test Rates run reads one channel at a time.
- **Reactive energy cannot be billed (finding 63).** kVArh is unit class 21, outside the classes a charge may use (the `UtilityChargeUnits` view allows 3, 4, 7 and 8), and the engine has no power factor or allowance math, so VED-GEN-IC's charge per kVArh beyond power factor 0.95 has no determinant. The older Rate Modeler's Power Factor Adjustment, which the product guide describes, has no counterpart in the current engine. The bills' reactive line comes from outside the engine, and so does the tax on that line.
- **A credit cannot move from one account's bill to another's (finding 64).** The engine prices one meter from that meter's own points and writes the result to that meter's own account (`UtilityRate_Exec`, `UtilityGenBill_Create`), and a bill adjustment applies only to charges priced in the same run. VED-BCT's transfer, out of the generator account and into the Central Plant's, is therefore not a rate; only its $12.50 fee is. Bill Allocation, the one feature that moves amounts between accounts, splits every line of a source meter's bill by percent and has no one-cycle lag.
- **A meter carries one rate at a time (finding 65).** The Tenant Rebill grid is a timeline: a row's End Date is the next row's Start Date, and the engine prices the row with the latest Start Date (`UtilityRateMeterAssignment_Get`, `UtilityRate_Exec`). A rider such as VED-BCT cannot run beside a base rate as the provider file lists it, so the two VED-BCT assignment rows are entered but do not compute beside their base rates, and the library's other multi-rate meters (the water rows among them) rest on the same assumption.
- **Bill Generation bills one meter at a time (finding 66).** Generation creates a bill on a meter's own account for each meter and period (`UtilityGenBill_Create`; `UtilityMeterRatesBillGen` holds one row per meter), so the ten turbine meters of one transfer agreement would get ten bills, the $1,250.00 research credit would post on each, and ten Transfer Energy lines rounded to the cent sum to $130,580.28, a cent over the hand-made statement's $130,580.27. The way round is the one EEM's own demo database uses, a logical billing meter whose only point is an aggregate of the other meters' points: Stage 2 adds the Ridgeline Farm Transfer Meter with the farm's Generated kWh aggregate, RWL-XFER-2024 is assigned to it alone and the turbines get no rate row, so the statement is one Transfer Energy line and one Research Credit. The rate itself reproduces the statement in one Test Rates run; whether generation does is for the first generated statement to show.

One correction to how the older lines in 05-rates.yaml are typed. A tax or bill adjustment multiplies its charges by the value as typed (`UtilityRates_Run_700`, `UtilityRates_Run_800`), and the product guide has the value entered as a decimal, so a 5% tax is 0.05, not 5.0. The older rates list their percentages as percent numbers (5.0, 2.85, -3.0, 1.9, 2.5, 1.75); type them as decimals (0.05, 0.0285, -0.03, 0.019, 0.025, 0.0175) and confirm the first Test Rates run. The Units box for those lines, like the fixed-charge lines, offers only the rate's own currency, and the generated charge name prints the decimal with a percent sign (finding 67).